

Programming Assignment 3: Baby Names

Due: Thursday, February 5th at 11pm CT

Learning Goals

- To implement individual classes based on given specifications and apply object-oriented programming principles, such as encapsulation
- To design and implement a program with multiple classes that interact with one another
- To practice using Java ArrayLists to efficiently store and process data
- To parse and process real-world data files with structured formats

Assignment Guidelines and Specifications

This is an individual assignment, which means you cannot work with a partner on this assignment. All code submitted must be entirely your own.

For this assignment, there are some additional specifications:

- You may use any classes and methods from the Java standard library that you choose.
 - You must use the `ArrayList` class as described below in the `NameRecord` class.
- You must add conditional statements to check the preconditions for public methods.
- You may (and are encouraged to) create private helper methods to remove redundancy and provide structure to your solution.
- Ensure your IDE does not add any non-standard Java imports for other Java classes called `Name` or `Names`. In the past, students have submitted code with bad imports and lost all correctness points due to compile errors. We recommend that you compile your `.java` files on the lab machines from the command line to ensure you do not have any non standard Java imports.
- **You may not share your tests with any other student for this assignment.**

Introduction

In this assignment, you will analyze [historical baby name popularity data](#), provided by the U.S. Social Security Administration. People have long been interested in analyzing names and their trends through the years (check out these articles and videos for a taste: [Where have all the Lisas Gone?](#), [How to Tell Someone's Age When All You Know Is Her Name](#), [The History of the Tiffany](#)), and this week, you will join the ranks of name-trend aficionados!

You will write a program that allows a user to query a database of the 1000 most popular baby names in the United States in a given time frame. The user will be able to look at interesting trends, such as names that surged in popularity during specific decades or those that consistently rose/declined in popularity over time.

You will implement three classes:

1. **NameRecord**: Represents a single name's ranking data.
2. **Names**: Manages a collection of **NameRecord** objects.
3. **NameSurfer**: Drives the user interface and menu system.

Getting Started

Here are the necessary files for this assignment:

- **NameRecord.java**: You will need to write this file entirely on your own, from scratch, according to the given specifications and submit this file.
- [Names.java](#): This file contains a class that manages a collection of **NameRecord** objects. You will need to complete this file according to the given specifications and submit this file.
- [NameSurfer.java](#): This file contains the main driver class for this assignment, along with significant starter code. You will need to complete this file according to the given specifications and submit this file.
- [names.txt](#), [names2.txt](#), [names4.txt](#): These files are example dataset files. More information about these files and their format is provided below. Do not submit these files.
- [nameSurferLog.txt](#): This file contains sample program output. The output from your **NameSurfer** should *roughly* match this example, though we will not be grading your **NameSurfer** output. Do not submit this file.

Dataset Files

Each dataset file has the following format:

- The first line specifies the base/starting decade (e.g., 1900). The base decade is the decade for the first rank.
- The second line specifies the number of decades the dataset covers (e.g., 11). This is the number of decades that each name is ranked for. You can assume that the number of decades will always be greater than or equal to 2. No dataset file will have only 0 or 1 decades.
- Each subsequent line contains a name, followed by rankings for each decade, in order. The names in the dataset are not guaranteed to be all lowercase, all uppercase, or properly capitalized. You can assume that all the names in a dataset file are unique.
- A ranking of 1 indicates that the name was the most popular name that year, while a ranking of 997 indicates that the name was not very popular.
- If a name was not ranked in the top 1000 names for a given decade, it is represented by a ranking of 0 (truthfully, it has some unknown rank greater than 1000).
- Every line in the file with data, including the last data line, ends with a newline character. This means that every file has a single blank line at the end.

For example, here are the first few lines of the names .txt file:

```
1900
11
A 83 140 228 286 426 612 486 577 836 0 0
Aa 100 0 0 0 100 200
Aaliyah 0 0 0 0 0 0 0 0 380 215
Aaron 193 208 218 274 279 232 132 36 32 31 41
```

The starting decade for this file is 1900, and every name should have a ranking for 11 decades (1900, 1910, 1920, ..., 2000).

The name 'A' was the 83rd most popular baby name in 1900, the 140th most popular in 1910, the 228th most popular in 1920, etc. In 1990 and 2000, the name 'A' was not in the top 1000 most popular baby names, hence the ranking of 0.

The name 'Aa' does not have 11 rankings. **If a line does not have the correct number of ranks based on the integer in the second line of the file, your program should ignore that line and name. Do not create a NameRecord object for that name.**

Your program should handle different starting decades and number of ranks correctly. Every dataset file will not start at the year 1900 or have 11 ranks per name. The starting decade may be before 1900 and may not be a multiple of 10. For example, a valid starting decade would be 1802. The starting decade could even be negative (this should not cause an exception to be thrown)!

A note for the curious: our data does not indicate if a name is a girl's name or a boy's name (unlike the U.S. SSA website). If a name appeared on both the list of boys' and girls' names for a given decade, the more popular rank was used in our dataset.

NameRecord

Implement a NameRecord class that represents a single name's ranking information and stores the associated data. Each NameRecord object should store (you **must** use the following data types):

1. The name (a String)
2. The base/starting decade (an int)
3. The ranking for each decade (an ArrayList of Integers)

Note: If a name has a ranking of 0 for a given decade, you may want to store something other than a 0 inside of your NameRecord to represent that the name is unranked in that decade. Of course, whatever you store will still need to be an Integer (in order to be an element in your ArrayList). You are not required to store an Integer other than 0; this note is merely a

suggestion. It may make some of your algorithms easier to implement, but it is not the singular "correct" way. There are always trade-offs involved for any design decision you make.

For the `NameRecord` class, you will be making a number of design decisions, including the instance variables, class variables, names of the methods, parameters for the methods, the preconditions of the methods, etc. You are required to have the following properties in your class:

1. A constructor to create a new `NameRecord`. You must decide what the appropriate parameters are for this constructor.
2. A method that returns the name for this `NameRecord`.
3. A method that returns the starting decade for this `NameRecord`.
4. A method that returns the total number of decades for this `NameRecord`, including, if applicable, the decades it is unranked.
5. A method that returns the name's rank for a given decade. The parameter for the given decade that is passed into this method will be an `int` from 0 to the (number of decades - 1) inclusive. For example, in `names.txt`, there is data for 11 decades. If the argument passed into this method is 0, the method should return the ranking for the start decade, 1900. If the parameter passed into this method is 1, the method should return the ranking in 1910. This method should return 0 if the name is unranked in the given decade (regardless of how you choose to store the unranked status for the `NameRecord` class).
6. A method that returns the name's most popular decade. In the case of a tie, return the more recent decade. This method should return the *actual* decade, not the index of the decade in the `ranks ArrayList`. For example, in `names.txt`, for Aaron, this method should return 1990, where it had its highest rank of 31. John's best rank was number 1, which occurred in 1900 and 1910. In this case, the method should return the more recent decade, 1910.
7. A method that returns the number of decades the name has been ranked in the top 1000.
8. A method that returns `true` if this name has been ranked in the top 1000 for every decade in the time period.
9. A method that returns `true` if this name has been ranked in the top 1000 in only one decade within the time period.

10. A method that returns `true` if this name has been getting more popular every decade in the time period covered. This will be true if every decade's rank is lower (closer to 1) than the previous decade. The rank must *improve* every decade; it cannot be equal to the previous decade. For example, the following rankings are always improving:

0 950 900 875 850 800 750 700 650 600 500 400

The following rankings are **not** always improving:

0 0 0 800 850 800 750 700 650 600 500 400

Recall that a ranking of 0 means that the name was outside of the top 1000 in that decade. When a rank of 0 appears twice consecutively, we do not have enough information to determine if the ranking improved during that time or not.

11. A method that returns `true` if this name has been getting less popular every decade in the time period covered. This will be true if every decade's rank is higher (further from 1) than the previous decade. The rank must *decline* every decade; it cannot be equal to the previous decade. For example, the following rankings are always declining:

100 150 200 202 250 300 350 400 450 460 0

The following rankings are **not** always declining:

100 150 200 202 250 300 350 400 0 0 0

12. A method that overrides the `toString` method from `Object`. The `String` representation of a `NameRecord` should be as follows:

```
<NAME>
<STARTING DECADE>: <RANK IN STARTING DECADE>
<STARTING DECADE+10>: <RANK IN STARTING DECADE+10>
...
<LAST DECADE>: <RANK IN LAST DECADE>
```

Each line in the returned `String` should have a newline character ("`\n`") at the end, including the very last line. If a name is not ranked in a given decade, the rank that is printed out should be 0, regardless of how you've chosen to store this rank in the object.

For example, for 'Aaron' from `names.txt`, the `String` representation would be:

```
Aaron
1900: 193
1910: 208
...
2000: 41
```

When writing the Names class in the next part of this assignment, you will see that the methods in that class that return lists of NameRecord objects require that the names be in sorted ascending order. The easiest way to meet this requirement is to have the NameRecord class implement the Comparable interface. We will discuss this in greater detail later in the semester, but for now, simply follow the instructions below:

1. Add the following clause to the class header:

```
public class NameRecord implements Comparable<NameRecord>
```

2. Implement the following instance method in the NameRecord class:

```
public int compareTo(NameRecord other)
```

In this method, you will compare two NameRecord objects by comparing the Strings within them that represent the names. The String object already has a compareTo method that you can use to compare two Strings. Therefore, simply call the String compareTo method on the calling object's name, pass in the name of the other NameRecord as the parameter, and return that result.

Testing NameRecord

Before moving on, you should thoroughly test the NameRecord class. Any issues that you do not catch now before moving on, will only cause bigger problems in the future. When testing, you can either use individual lines from the dataset files we have provided or create your own testing data.

Write your testing code in NameSurfer.java. You are expected to write 2 test cases for every method in NameRecord, except the constructor and the compareTo method. In other words, you will be writing test cases for the methods we have numbered 2 through 12 above.

Note: For this assignment, you may **not** share your tests with any other students.

Names

Finish implementing the Names class, which manages a collection of NameRecord objects. This class should store all of the NameRecord objects in an ArrayList:

```
private ArrayList<NameRecord> names
```

Note: All methods that return an ArrayList of NameRecords or an ArrayList of Strings must return the lists in sorted ascending order based on the names.

1. Implement the following constructor:

```
public Names(Scanner sc)
```

This constructor accepts a `Scanner` object as a parameter that is already connected to a dataset file. In the current starter code in `NameSurfer.java`, you can see that the `Scanner` is currently connected to `names.txt`.

In this constructor, you should use the `Scanner` to read and save the starting decade and number of decades from the connected dataset file. Then, use the `Scanner` to read through all the names in the file and create a `NameRecord` object for each one. You can read in the name data with the following code:

```
while (sc.hasNextLine()) {  
    String line = sc.nextLine();  
    // Create a NameRecord object based on line  
    // Add the object to the ArrayList of NameRecord objects  
}
```

You will need to parse each line you read into the name and the rankings. There are two ways we recommend doing this. The first approach uses the [split method](#) from the `String` class (where `line` is a single line with a name and rankings read in from the file):

```
String[] parsedData = data.split("\\s+");
```

This will parse the `String` into its individual components, using whitespace as the delimiter. The name itself will be the first element of the resulting array and the ranks will be in the remaining elements. It is important to note that the rankings will be saved in the array as `Strings`, not `ints`. You must then convert the rankings to `ints` using the [Integer.parseInt](#) method.

The second approach includes using another `Scanner` object to read each line. First, create a [Scanner object](#) connected to `line` (where `line` is a single line with a name and rankings read in from the file):

```
Scanner lineScanner = new Scanner(line);
```

Now, use the `Scanner`'s `next` method to pull out the name. Then, use the `nextInt` method to pull out each of the rankings.

If a name is ranked outside the top 1000 for every decade (if it has all 0s for rankings), your program should ignore that line and name. Do not create a `NameRecord` object

for that name. Additionally, if a line does not have the correct number of rankings, do not create a `NameRecord` object for that line.

Note: After constructing the `ArrayList` of `NameRecord` objects you may find it useful to sort the list. You can do so by calling the [Collections.sort\(\) method](#).

2. Implement the following method:

```
public NameRecord getName(String name)
```

This method returns the `NameRecord` whose name is equal to the given parameter, `name`, ignoring case. If there is no `NameRecord` that matches the parameter, return `null`.

3. Implement the following method:

```
public ArrayList<NameRecord> getMatches(String partialName)
```

This method returns an `ArrayList` of `NameRecord` objects that contain the given substring parameter, `partialName`, ignoring case. The returned `NameRecords` in the `ArrayList` must be in sorted order based on the names.

4. Implement the following method:

```
public ArrayList<String> rankedEveryDecade()
```

This method returns an `ArrayList` of `Strings` of names that have been ranked in the top 1000 names or better every decade. The returned `Strings` in the `ArrayList` must be in sorted order based on the names.

5. Implement the following method:

```
public ArrayList<String> rankedOnlyOneDecade()
```

This method returns an `ArrayList` of `Strings` of names that have been ranked in the top 1000 names or better in exactly one decade. The returned `Strings` in the `ArrayList` must be in sorted order based on the names.

6. Implement the following method:

```
public ArrayList<String> alwaysMorePopular()
```

This method returns an `ArrayList` of `Strings` of names that have been getting more popular every decade. The returned `Strings` in the `ArrayList` must be in sorted order based on the names.

7. Implement the following method:

```
public ArrayList<String> alwaysLessPopular()
```

This method returns an `ArrayList` of `Strings` of names that have been getting less popular every decade. The returned `Strings` in the `ArrayList` must be in sorted order based on the names.

8. Implement a method of your own that checks for names that meet some criteria of your own design. For example, you might have found all the names that can be written using symbols from the periodic table, or all the names that staged a comeback (were in the top 1000, dropped out for some time, and then made a reappearance). Of course, you can't use these exact ideas, but hopefully this helps you get some ideas going!

You must include thorough documentation of what your criteria is in the method comments. Your method will be graded based on its originality and associated documentation.

Note: You do not need to submit your own test cases for the `Names` class.

NameSurfer

The `NameSurfer` class contains some simple tests and the main driver for your program.

Use the `simpleTest` method to test your `Names` and `NameRecord` classes. Change the `NAME_FILE` in order to test on the other dataset files that we have provided to you, or on your own files that you created. Before submitting, delete the call to `simpleTest` in `main`.

Note: You do not need to submit your own test cases for the `NameSurfer` class.

The `main` method also calls `runOptions`, which presents the different menu choices to the user and calls the associated methods. You must complete these associated methods according to their descriptions in their comments. The code to check the preconditions of these methods has already been called. You should call your novel method that you designed as the `STUDENT_SEARCH` option in the menu. Additionally, at the top of the file, explain your method under the associated comment.

For expected program behavior, review the [sample run log](#). Your output should match that of the sample run log, with two exceptions: 1) do not trim the output for operations that result in a large number of names, and 2) include the description of the search you developed. **We are not grading based on the output of `NameSurfer`, so do not worry about minor differences in the output of your `NameSurfer` and what is shown in the log.** Rather, we will grade your assignment by calling methods from your `Names` class and checking those results.

Now that your program is entirely finished, you can investigate various trends in naming children over the decades. In a comment at the top of the file, discuss one interesting trend you found and back it up with data/results. Here are some examples (of course, you may not use these as your interesting trend):

- Why do you think Rock was popular in the 1950s and Trinity in the 2000s?
- Search for your grandparents' names. Some of the names may sound old-fashioned, but wait long enough and they might stage a comeback. Why is that?
- Try historical names like Sigmund, Adolf, and Ike. What happened with variations such as Adolph?
- Why are some letters listed as names? A? M? J? Note that In a more recent version of the database single letter names are not included.

Submission Checklist

Before you submit, run through our submission checklist to make sure you didn't forget anything! Did you remember to:

1. Create your own NameRecord class and implement all the required methods?
2. Include the required test cases for NameRecord in NameSurfer.java?
3. Implement all the required methods in the Names class?
4. Implement all the required methods in the NameSurfer class?
5. Add your own menu option and document it at the top of the NameSurfer class?
6. Comment on an interesting trend you found at the top of the NameSurfer class?
7. Fill in the header comments for all three files?
8. Check the preconditions in public methods and throw exceptions as necessary?
9. Follow the [program hygiene guide](#)?
10. Follow the assignment specifications from page 3 of the [Assignment handout](#)?

If you have done all these things, then you're ready to submit NameRecord.java, Names.java and NameSurfer.java to Gradescope!

*Thank you to Professor Mike Scott for creating the previous version of this assignment!
Based on an assignment created by Nick Parlante and Stuart Reges.*